

Guion de exposición — AgroNomad

Duración sugerida: 5 a 7 minutos.

Idea central: AgroNomad conecta un collar de monitoreo con una estación local y una interfaz visual para centralizar ubicación y temperatura del ganado.

Las frases entre comillas son para decir en voz alta. Las notas marcadas como **Nota para el equipo** son contexto técnico y no hace falta leerlas literalmente.

1. Elevator pitch — 30 segundos

“AgroNomad es un sistema de monitoreo de ganado pensado para ayudar al productor a saber dónde está cada animal y consultar datos tomados por un collar inteligente. El collar AgroNeck obtiene ubicación GPS, temperatura y movimiento; transmite sus lecturas por radio LoRa a una estación fija llamada AgroBot. Desde esa estación, los datos se guardan y se muestran en una interfaz web, donde se puede ver el rodeo sobre un mapa. El proyecto integra electrónica, firmware, comunicaciones y software en una solución modular que buscamos hacer autónoma y útil en el campo.”

2. Apertura y problema — alrededor de 45 segundos

“En un campo, controlar visualmente a cada animal requiere tiempo y no siempre permite conocer su ubicación o detectar cambios entre recorridas. AgroNomad propone complementar ese trabajo con tecnología: identificar los animales, reunir información desde el campo y consultarla en una estación central. No reemplaza al productor ni al veterinario; busca aportar datos para que el seguimiento y la toma de decisiones sean más ágiles.”

3. Explicación sencilla: las tres partes — alrededor de 1 minuto

“El sistema se entiende con tres piezas. Primero está el AgroNeck, el collar que lleva el animal y reúne lecturas de sensores. Segundo está el AgroBot, una estación receptora basada en una Raspberry Pi que escucha las transmisiones de los collares. Tercero está la aplicación web, que muestra los animales y sus últimas ubicaciones en un mapa.

La secuencia es: el collar mide, transmite por LoRa, el AgroBot recibe y envía la lectura al servidor; el servidor la guarda y la interfaz la presenta. Así, una lectura que nace en el animal puede terminar visible en una computadora o un teléfono conectado a la estación.”

Analogía breve: el AgroNeck es el “reportero” que lleva el animal; LoRa es el “correo de radio”; el AgroBot es la “central de recepción”; y la aplicación es el “tablero” donde se consulta la información.

4. Cómo funciona técnicamente — alrededor de 2 minutos

“En el collar, una ESP32 coordina los módulos. El GPS NEO-6M entrega coordenadas por UART. El MPU6050 mide aceleración y giro, y además se usa para detectar movimiento y despertar al dispositivo. El MLX90614 mide temperatura sin contacto, y el LM35 aporta una lectura de temperatura ambiente por una entrada analógica. El firmware calcula una temperatura estimada, empaqueta el identificador, la temperatura y las coordenadas en una estructura binaria de 14 bytes y la transmite con un SX1278 por LoRa.

La estación AgroBot usa una Raspberry Pi Zero 2 W y otro SX1278. El receptor se comunica con la radio por SPI y detecta la llegada de paquetes mediante la señal DIO0. Cuando recibe una trama, verifica el tamaño y el CRC de radio, interpreta los 14 bytes y los convierte a JSON. Luego los envía por TCP al backend local.

El backend tiene dos canales: HTTP para las consultas de la aplicación y TCP para recibir telemetría. El parser reconstruye los objetos JSON aunque los datos lleguen fragmentados o varios mensajes juntos, normaliza campos como ID, latitud, longitud y temperatura, y guarda la lectura en PostgreSQL

con PostGIS. La base conserva posiciones con fecha y hora; para el mapa, la API devuelve la última posición disponible.

Por último, el frontend está hecho con React y Leaflet. Se conecta al backend por WebSocket para recibir actualizaciones de animales, dibuja los marcadores y los límites de los potreros, y permite seleccionar un animal y cambiar su nombre.”

5. Electrónica, energía y diseño físico — alrededor de 1 minuto

“En electrónica, la placa Cortex reúne la ESP32 y las conexiones para GPS, LoRa y sensores. Los sensores MPU6050 y MLX90614 comparten el bus I2C; el GPS usa UART; LoRa utiliza SPI; y el LM35 se lee por ADC. La placa AgroBot adapta la Raspberry Pi y el módulo SX1278 para recibir las transmisiones.

El diseño también incluye placas auxiliares: LoraShield adapta el módulo de radio, MLXModule facilita la conexión del sensor infrarrojo y MOSFETAdapter adapta MOSFETs de montaje superficial. Solaria reúne la parte de alimentación solar, carga y regulación de tensión. Las carcasas de AgroNeck y AgroBot tienen modelos 3D y archivos STEP para revisar su forma y montaje.”

Nota para el equipo: en el firmware del AgroNeck están definidas estas conexiones: GPS RX/TX en GPIO 16/17, I2C en GPIO 21/22, interrupción del MPU6050 en GPIO 27 y entrada del LM35 en GPIO 34. LoRa está definido por SPI en GPIO 5, 14, 18, 19 y 23, con DIO0 en GPIO 10. Confirmar el pinout físico de la placa antes de presentarlo como montaje validado.

6. Qué muestra hoy el software — alrededor de 45 segundos

“La pantalla principal permite ver el mapa satelital, los animales que tienen una posición válida, sus límites de campo, el identificador, el nombre y la temperatura recibida. También se puede renombrar un animal. El sistema está organizado para separar la interfaz, la API, la recepción de datos y la base espacial, de modo que cada parte pueda evolucionar sin rehacer todo el conjunto.”

7. Estado actual y próximos pasos — alrededor de 1 minuto

“El repositorio ya contiene firmware para la ESP32, el receptor LoRa para Raspberry Pi, un backend con PostgreSQL/PostGIS, una interfaz web y los diseños electrónicos y mecánicos. También hay una guía de prueba de extremo a extremo.

Como próximos pasos, necesitamos validar el enlace de radio y la autonomía en el montaje real, consolidar el manejo de energía, calibrar las mediciones y ampliar la seguridad de las comunicaciones. Después se puede completar el historial conectado a datos reales, las alertas y las funciones de administración del campo. La meta es avanzar desde un prototipo integrado hacia una herramienta confiable de monitoreo.”

8. Cierre — 15 segundos

“En resumen, AgroNomad transforma lecturas del campo en información visible: el AgroNeck mide, el AgroBot recibe y el software organiza los datos. El valor del proyecto está en integrar esas capas en un sistema modular, de bajo consumo y pensado para las necesidades de la actividad ganadera.”

Mapa del repositorio para responder preguntas

Carpeta o archivo	Qué contiene y cómo explicarlo
README.md	Presentación general, propósito, componentes y estructura del proyecto.
docs/	Anteproyecto, informes, presentaciones y carpeta de campo. Sirve para explicar el origen, objetivos y
firmware/	Firmware de los dispositivos y software de la estación.

Carpeta o archivo	Qué contiene y cómo explicarlo
firmware/AgroNeck/	Aplicación ESP-IDF de la ESP32: sensores, GPS, LoRa, activación por movimiento y modos de bajo consumo.
firmware/AgroBot/	Backend, frontend, programas C de recepción/prueba y guía de montaje y prueba.
hardware/	Esquemáticos, PCB, bibliotecas y renders de las placas. Incluye AgroBot, Cortex, LoraShield, MLXM.
mechanical/	Renders y modelos STEP de las carcasas de AgroNeck y AgroBot.
web/	Landing page pública para presentar el proyecto; no es el panel operativo que consulta la telemetría.

Archivos de software especialmente útiles

- [AgroNeck main.c](#): secuencia de lectura, cálculo, transmisión LoRa, detección de movimiento y deep sleep.
- [lora.c](#) del [AgroNeck](#): configuración SPI/SX1278 y envío de la estructura binaria.
- [mpumlx.c](#): inicialización I2C, lecturas del MPU6050/MLX90614 y detección Wake-On-Motion.
- [neo6m.c](#): lectura UART y conversión de coordenadas NMEA.
- [lm35.c](#): lectura ADC y conversión de tensión a temperatura.
- [lora_receiver.c](#): recepción SX1278 en Raspberry, decodificación de la trama y reenvío JSON por TCP.
- [server.js](#): arranque del backend HTTP, TCP y WebSocket.
- [telemetryServer.js](#) y [telemetryParser.js](#): recepción y procesamiento de telemetría.
- [db/index.js](#) y [schema.sql](#): persistencia, consultas y esquema PostgreSQL/PostGIS.
- [useAnimalDashboard.js](#), [Home.jsx](#) y [AnimalMap.jsx](#): carga de datos, estado del tablero y visualización del mapa.
- [Setup.md](#): orden recomendado para desplegar y probar el flujo completo.

Precisión importante: implementación frente a visión

Conviene hablar de “prototipo” o “función en desarrollo” cuando corresponda. El repositorio combina documentación de visión del producto con módulos ya implementados; no todo lo que aparece en la landing page o en las pantallas está conectado a telemetría real.

- **Pulso y frecuencia cardíaca:** aparecen en algunos textos de presentación, pero el firmware revisado no contiene un sensor cardíaco ni calcula o transmite pulsaciones. El MPU6050 es un acelerómetro/giroscopio usado para movimiento, no un sensor de pulso.
- **Datos enviados actualmente:** la trama binaria contiene ID, temperatura estimada y coordenadas GPS. No contiene aceleración, velocidad, pulso ni estadísticas diarias.
- **Temperatura:** el firmware lee el MLX90614 y el LM35 y calcula un valor estimado. No conviene presentarlo como diagnóstico clínico ni como medición validada de temperatura corporal sin calibración experimental.
- **Radio:** el firmware transmisor y [lora_receiver.c](#) del AgroBot están configurados para **433 MHz**; la guía [Setup.md](#) también indica 433 MHz. Sin embargo, el README de ejemplo de firmware del AgroBot menciona **915 MHz**. Hay que unificar la documentación y confirmar módulo, antena, banda permitida y configuración antes de una demostración de campo.
- **Cifrado y seguridad:** el README del AgroNeck deja el cifrado como trabajo pendiente. La trama no debe presentarse como cifrada.
- **Bajo consumo y energía solar:** el firmware incluye deep sleep y configuración de Wake-On-Motion; la placa Solaria y la lógica de MOSFETs están en el diseño. La conmutación y la autonomía deben describirse como integración a validar, no como rendimiento ya medido.
- **Actualización de pantalla:** el README del frontend dice 15 segundos, pero el backend WebSocket consulta y difunde actualmente cada 1 segundo. Para exponer, es más preciso decir “actualización en vivo por WebSocket” que prometer un período fijo de refresco.
- **Pantallas de demostración:** los datos de actividad, alertas, métricas generales, perfil de usuario e historial que aparecen en algunas vistas son datos fijos de ejemplo. La página de historial no consume un endpoint de trayectorias; el panel de configuración de campos todavía es muy básico.
- **Inicio de sesión:** existe una pantalla de login, pero el backend revisado no expone una ruta de autenticación ni gestiona sesiones. No presentarlo como autenticación funcional.
- **Conectividad:** el trayecto collar-estación usa LoRa y la estación habla con el backend local por TCP. La interfaz usa HTTP/WebSocket; el mapa satelital necesita acceso a las teselas del proveedor de mapas. No

afirmar conectividad celular/GSM implementada.

Guía rápida de vocabulario

- **ESP32:** microcontrolador que coordina las lecturas y el envío desde el collar.
- **GPS / NEO-6M:** receptor que obtiene latitud y longitud desde satélites.
- **MPU6050:** sensor de aceleración y giro; en este prototipo ayuda a detectar movimiento.
- **MLX90614:** sensor infrarrojo de temperatura sin contacto.
- **LM35:** sensor analógico de temperatura ambiente.
- **LoRa / SX1278:** radio de largo alcance y bajo consumo para enviar paquetes pequeños; el alcance real depende de antenas, obstáculos, potencia y normativa.
- **SPI, I2C, UART y ADC:** buses/interfaz entre módulos: radio, sensores, GPS y lectura analógica, respectivamente.
- **PostgreSQL/PostGIS:** base de datos relacional con tipos y funciones para información geográfica.
- **WebSocket:** conexión persistente para que el servidor envíe novedades al navegador.
- **Deep sleep:** modo de consumo reducido de la ESP32; el MPU6050 puede generar la señal que despierta al collar por movimiento.